Group 60 Progress Report:
2048 Machine learning model to determine the next best move

Huaihui Zeng, Suzanne Abdullah
{zengh22,abduls61}@mcmaster.ca

1 Introduction (DONE)

We used an existing algorithm to play 2048 and collected the board state before each recorded move and the move it chose as training data. We trained the model on these data without explicitly giving it the algorithm's rules. We aim to find out whether the model can learn strategies and imitate the algorithm using only examples of board states and chosen moves.

2 Related Work

Machine learning models that aim to select moves in 2048 resulting in optimal moves are not uncommon, and many have been created prior to ours. Our biggest reference for this project was the github repository the project used for data collection (game difficulty et al., 2024). This system used a deep-knowledge machine learning model to create their algorithms. Our key takeaway from the strategy used in this repository was that a lack of monotonicity in the model can help greatly in creating a model that can surpass other 2048 AIs, as the engines flexibility allows it to fix dynamically fix mistakes. This repository used a reinforcement learning model, which made sense considering the nature of the problem. The penalty for reaching death states shifts dynamically in this model, which might be a good strategy for future iterations fo the model. This model reached very high levels of success, reaching a 100 percent success rate over 8k moves. Two other models (Chan, 2022) (Fuchs) were first created using a random direction choice, and then was rewarded in increasingly complex manors depending on the models progress in the game. These models also used a neurel network strategy, but on a much simpler level than the github repository, as only one hidden layer was used during this implementation. This article by Fuchs greatly informed our method of feature classification, as we used a similar framework in storing moves to the one described here. This article also uniquely introduces a 'penalty' for illegal moves, which could be useful in trying to filter out unnecessary behaviour in the model early on. A dissertation by Dr. Guei (Guei, 2023) breaks down multiple strategies that can be employed for a reinforcement learning approach to the game of 2048. This article provides a lot of details about the results of multiple strategies for training a 2048, including ensemble learning, Monte Carlo tree search and deep reinforcement learning. This article is very intensive, and will be very useful to our team moving forward with different strategies during the rest of the project. The investigations into 'greedy' algorithms was especially interesting, as our team had yset to consider their application potential in the project. Finally, an article from 2016 (Nie et al., 2016) was investigated, which used a minimax and expectimax search tree strategy, that were improved upon via human heuristics. This paper also used reinforcement learning, to some degree of success after pruning, which showed our team that a tree strategy could be fruitful in future implementations. Overall, our biggest takeaways from our research were that reinforcement strategies were the most effective, and larger datasets were more ideal in getting more accurate results.

3 Dataset (DONE)

Our dataset was collected using a scraping technique to generate games from the front-end implementation of the existing 2048 ML model our program utilizes: https://game-difficulty.github.io/2048EndgameTablebase/. Using Selenium and Beautifulsoup libraries, our team created a program that would open the 2048 simulator in a browser window and increment through games one turn at a time, saving the board state and current score on each move. The boardstate was collected from the corresponding hexadecimal code

given at the top of the page. This code was split into individual board tiles (1 digit for each tile), and each tile was converted into its corresponding decimal value. Each board tile was stored in the logarithmic form (e.g. 4 would be stored as 2). Each board tiles name was adherent to the A-D:1-4 formatting, with the letter representing the row, and the number representing the column. At the end of a game, the board would be cleared and reset to a valid starting state that was randomly generated by the program. The dataset also includes an estimate for what move was made, based off of the board state before and after each move was made. When a valid estimate for the previous move could not be made, the program defaulted to a 0 value for the move value. These zeroes were filtered out in the preprocessing stage. The program exports collected data to an excel sheet, and multiple datasheets were created with varying amounts of games and turns.

4 Features (DONE)

We represented each 4×4 board as 16 numbers, one for each cell, in row order. An empty cell was encoded as 0, and a tile with value 2^n was encoded as n. We kept all 16 features and did not use data augmentation. We fitted `StandardScaler` on the training data and used it to standardize both the training and test data before running k-NN.

5 Implementation (DONE)

We used scikit-learn to implement a k-nearest neighbors (k-NN)classifier with $k = 5$. The model takes the 16 encoded tile values from a board and predicts one of four moves: up, down, left, or right. We calculated the scaling statistics from the training boards and used them to standardize both the training and test boards.

For each test board, the classifier finds the five closest training boards using Euclidean distance. Each neighbor gets one vote, and the move with the most votes is the prediction. Our k-NN model stores the training boards and predicts by voting among the five closest ones, so it does not need a loss function during training.

For the baseline, we ranked the four moves by how often they appeared in the training data. For each test board, the baseline checks the moves in that order and picks the first one that slides or merges a tile.

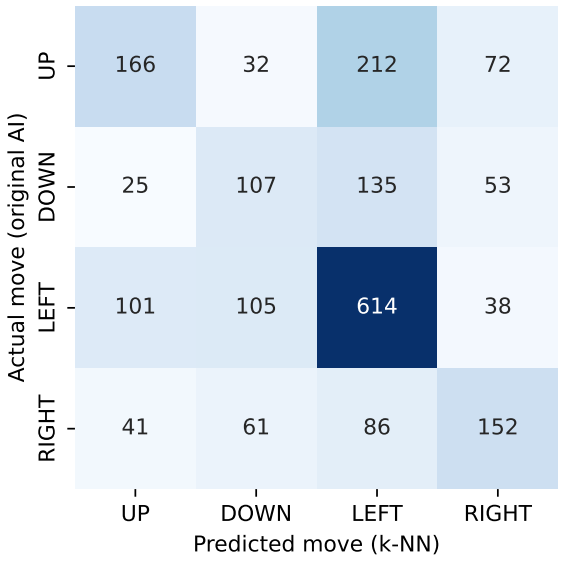


Figure 1: Confusion Matrix showing 52% accuracy

6 Results and Evaluation (DONE)

We split the data by game, using the first 80% of games for training and the remaining 20% for testing. We did not use a separate validation set or cross-validation. Each game contributed 50 recorded moves. We measured test accuracy as the percentage of predictions that matched the original AI's moves. For each test board, the baseline chose the legal move that appeared most often in the training data.

As shown in Table 1, k-NN beat the baseline in four of the five experiments. In the 200-game experiment, k-NN achieved 52.0% accuracy, compared with 47.7% for the baseline.

Games	k-NN($k = 5$)	Baseline
5	48.0%	66.0%
10	49.0%	41.0%
50	54.8%	45.4%
135	52.4%	51.0%
200	52.0%	47.7%

Table 1: Move prediction accuracy by dataset size

7 Feedback and Plans

Write about your plans for the remainder of the project. This should include a discussion of the feedback you received from your TA, and how you plan to improve your approach. Reflect on your implementation and areas for improvement. Refer to item 6. This may be around 0.5 pages.

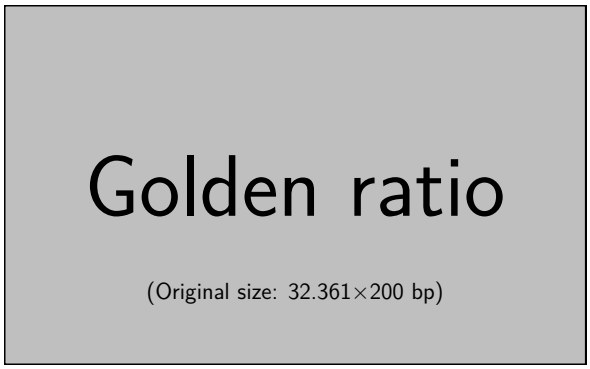


Figure 2: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

8 Template Notes

You can remove this section or comment it out, as it only contains instructions for how to use this template. You may use subsections in your document as you find appropriate.

8.1 Tables and figures

See Table 2 for an example of a table and its caption. See Figure 2 for an example of a figure and its caption.

8.2 Citations

Table 2 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to get "author (year)" citations, like this citation to a paper by Gusfield (1997). You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations (Gusfield, 1997). You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. Gusfield, 1997).

8.3 References

Many websites where you can find academic papers also allow you to export a bib file for citation or bib formatted entry. Copy this into the `custom.bib` and you will be able to cite the paper in the LaTeX. You can remove the example entries.

8.4 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{1}$$

Labels for equation numbers, sections, subsections, figures and tables are all defined with the `\label{label}` command and cross references to them are made with the `\ref{label}` command. This an example cross-reference to Equation 1. You can also write equations inline, like this: $A = \pi r^2$.

Team Contributions

Suzanne worked on the data collection program for the ML model, while Huaihui worked on the ML model itself. The sections of the report were split between team members. Huaihui completed the introduction, model inputs and implementation and evaluation strategies, while Suzanne completed the related work, dataset and preprocessing, TA feedback, and team contribution sections.

References

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.

Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.

Lok Hin Chan. 2022. Playing 2048 with deep q-learning (with pytorch implementation).

Sam Fuchs. 2048 ai.

game difficulty, Narwhal, and Sitiyou. 2024. 2048endgametablebase.

Hung Guei. 2023. *On Reinforcement Learning for the Game of 2048*. Ph.D. thesis.

Dan Gusfield. 1997. *Algorithms on Strings, Trees and Sequences*. Cambridge University Press, Cambridge, UK.

Yun Nie, Wenqi Hou, and Yicheng an. 2016. Ai plays 2048. *Stanford Edu*.

Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. Yara parser: A fast and accurate dependency parser. *Computing Research Repository*, arXiv:1503.06733. Version 2.

Output	natbib command	ACL only command
(Gusfield, 1997)	\citep	
Gusfield, 1997	\citealp	
Gusfield (1997)	\citet	
(1997)	\citeyearpar	
Gusfield's (1997)		\citeposs

Table 2: Citation commands supported by the style file.